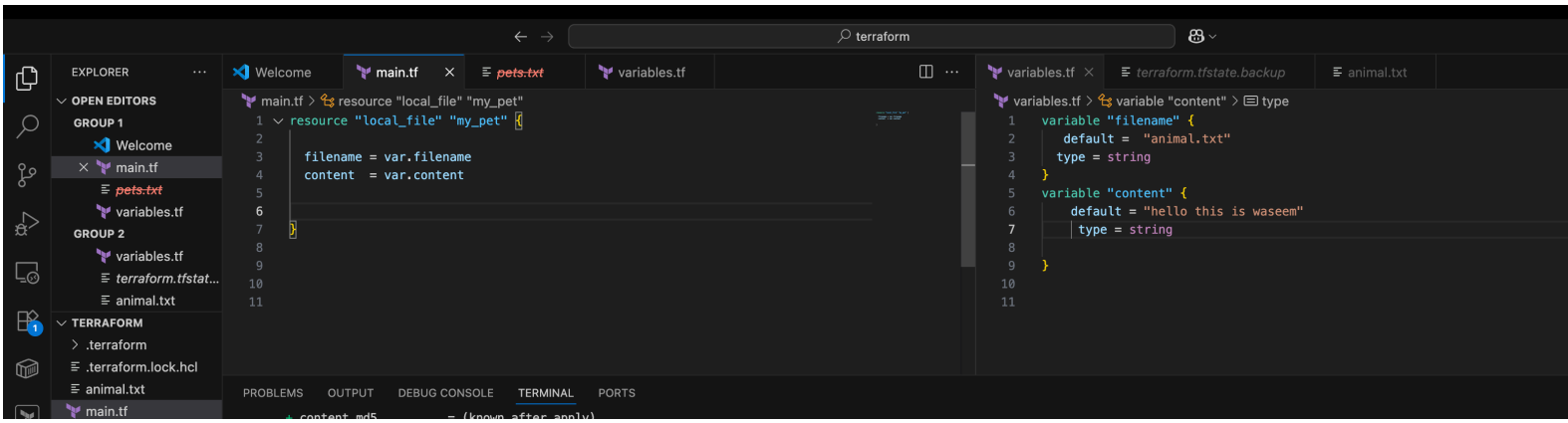
Terraform 3

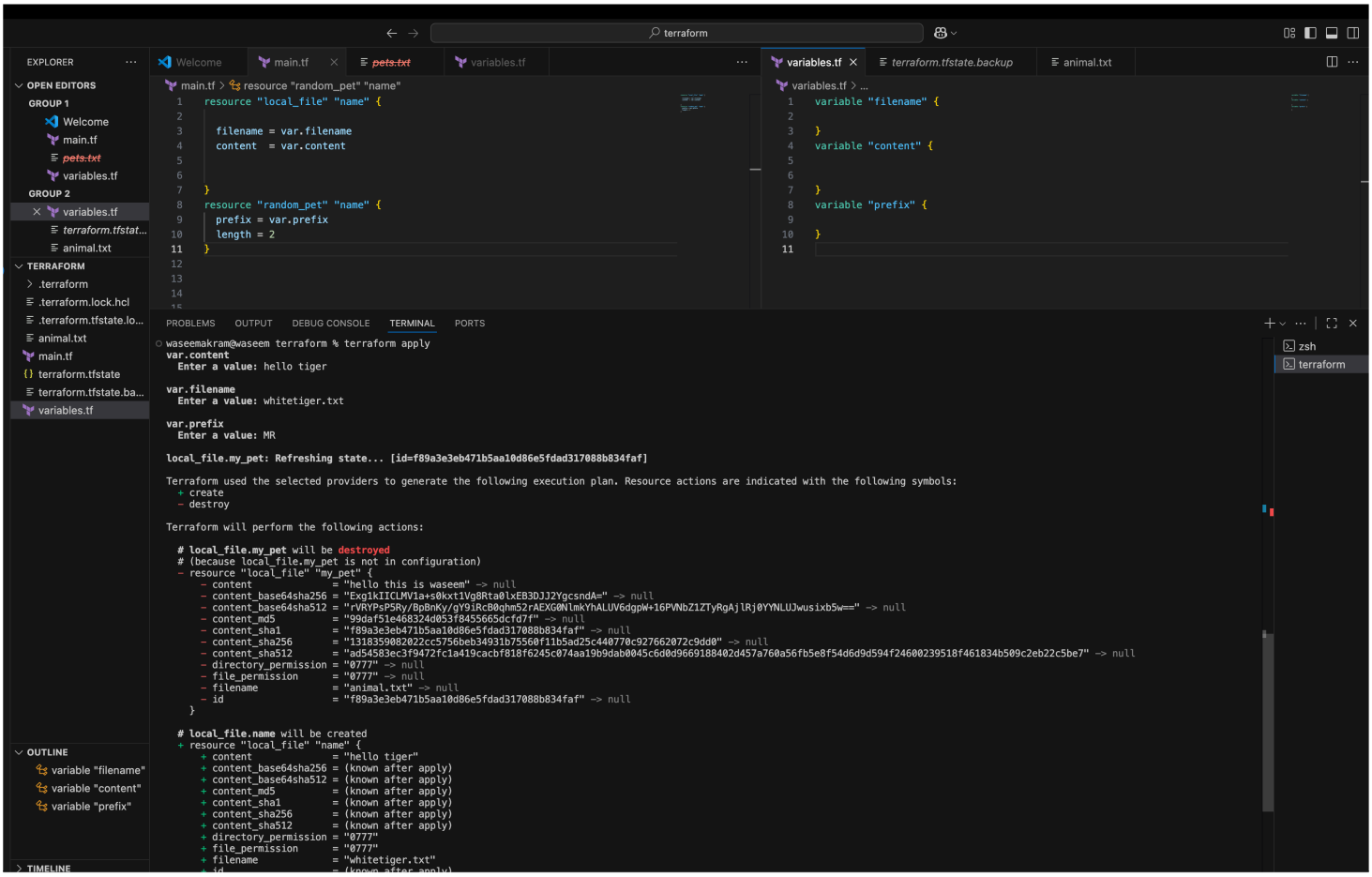
1. **Watch the Terraform-03 video.**

2. **Execute the script shown in the video.**

Using of variables:
==========
1) By using varibles.tf file



2) By using interactive mode (This will get activated if we dont pass default value in variable.tf file)

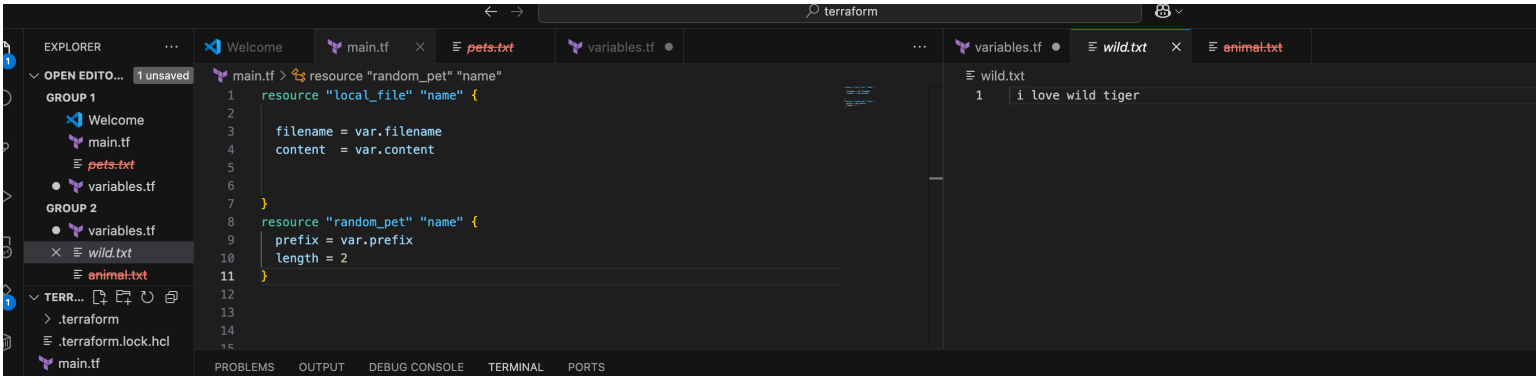



```
waseemakram@waseem terraform % terraform apply
var.content
  Enter a value: hello tiger

var.filename
  Enter a value: whitetiger.txt

var.prefix
  Enter a value: MR

local_file.my_pet: Refreshing state... [id=f89a3e3eb471b5aa10d86e5fdad317088b834faf]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create
  - destroy

Terraform will perform the following actions:

  # local_file.my_pet will be destroyed
  # (because local_file.my_pet is not in configuration)
  - resource "local_file" "my_pet" {
      - content              = "hello this is waseem" -> null
      - content_base64sha256 = "Exg1kIICLMV1a+s0kxt1Vg8Rta0lxEB3DJJ2YgcsndA=" -> null
      - content_base64sha512 = "rVRYPsP5Ry/BpBnKy/gY9iRcB0qhm52rAEXG0NlmkYhALUV6dgpW+16PVNbZ1ZTyRgAjlRj0YYNLUJwusixb5w==" -> null
      - content_md5          = "99daf51e468324d053f8455665dcfd7f" -> null
      - content_sha1         = "f89a3e3eb471b5aa10d86e5fdad317088b834faf" -> null
      - content_sha256       = "1318359082022cc5756beb34931b75560f11b5ad25c440770c927662072c9dd0" -> null
      - content_sha512       = "ad54583ec3f9472fc1a419cacbf818f6245c074aa19b9dab0045c6d0d9669188402d457a760a56fb5e8f54d6d9d594f24600239518f461834b509c2eb22c5be7" -> null
      - directory_permission = "0777" -> null
      - file_permission      = "0777" -> null
      - filename             = "animal.txt" -> null
      - id                   = "f89a3e3eb471b5aa10d86e5fdad317088b834faf" -> null
    }

  # local_file.name will be created
  + resource "local_file" "name" {
      + content              = "hello tiger"
      + content_base64sha256 = (known after apply)
      + content_base64sha512 = (known after apply)
      + content_md5          = (known after apply)
      + content_sha1         = (known after apply)
```

3) Command line flags

-->

terraform apply -var "filename=wild.txt" -var "prefix=MR" -var "content= i love wild tigers"




```
waseemakram@waseem terraform % terraform apply -var "filename=wild.txt" -var "prefix=MR" -var "content= i love wild tiger"

local_file.name: Refreshing state... [id=562a1098d3e8a3470649ed3f47a8488bfe8ada45]
random_pet.name: Refreshing state... [id=MR-optimal-snapper]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

  # local_file.name must be replaced
-/+ resource "local_file" "name" {
      ~ content              = "hello tiger" -> " i love wild tiger" # forces replacement
      ~ content_base64sha256 = "pRxBC6D14/FqIxMDRqsTi2xGDDGnKU1Jw/8DShqu7uQ=" -> (known after apply)
      ~ content_base64sha512 = "nWgXK+45kkI3sbOdsYUIGUp2bQ06SmwVY3y07/2Iwduy2C+ELBSZ8FqR9vCO4FmG0iA3wM1d9uBgM3PjTPhcVw==" -> (known after apply)
      ~ content_md5          = "6bd9329556e5cd27ab801f6c1a4fb6e2" -> (known after apply)
      ~ content_sha1         = "562a1098d3e8a3470649ed3f47a8488bfe8ada45" -> (known after apply)
      ~ content_sha256       = "a51c410ba0f5e3f16a23130346ab138b6c460c31a7294d49c3ff034a1aaeeee4" -> (known after apply)
      ~ content_sha512       = "9d68172bee39924237b1b39db18508194a766d0d3a4a6c15637cb4effd88c1dbb2d82f842c1499f05a91f6f08ee05986d22037c0cd5df6e0603373e34cf85c57" -> (known after app
      ~ filename             = "whitetiger.txt" -> "wild.txt" # forces replacement
      ~ id                   = "562a1098d3e8a3470649ed3f47a8488bfe8ada45" -> (known after apply)
        # (2 unchanged attributes hidden)
    }

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.name: Destroying... [id=562a1098d3e8a3470649ed3f47a8488bfe8ada45]
local_file.name: Destruction complete after 0s
local_file.name: Creating...
local_file.name: Creation complete after 0s [id=3fde2e0b5d5528ef88db2502236d450252e22253]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
waseemakram@waseem terraform %
```

4) Environment variables

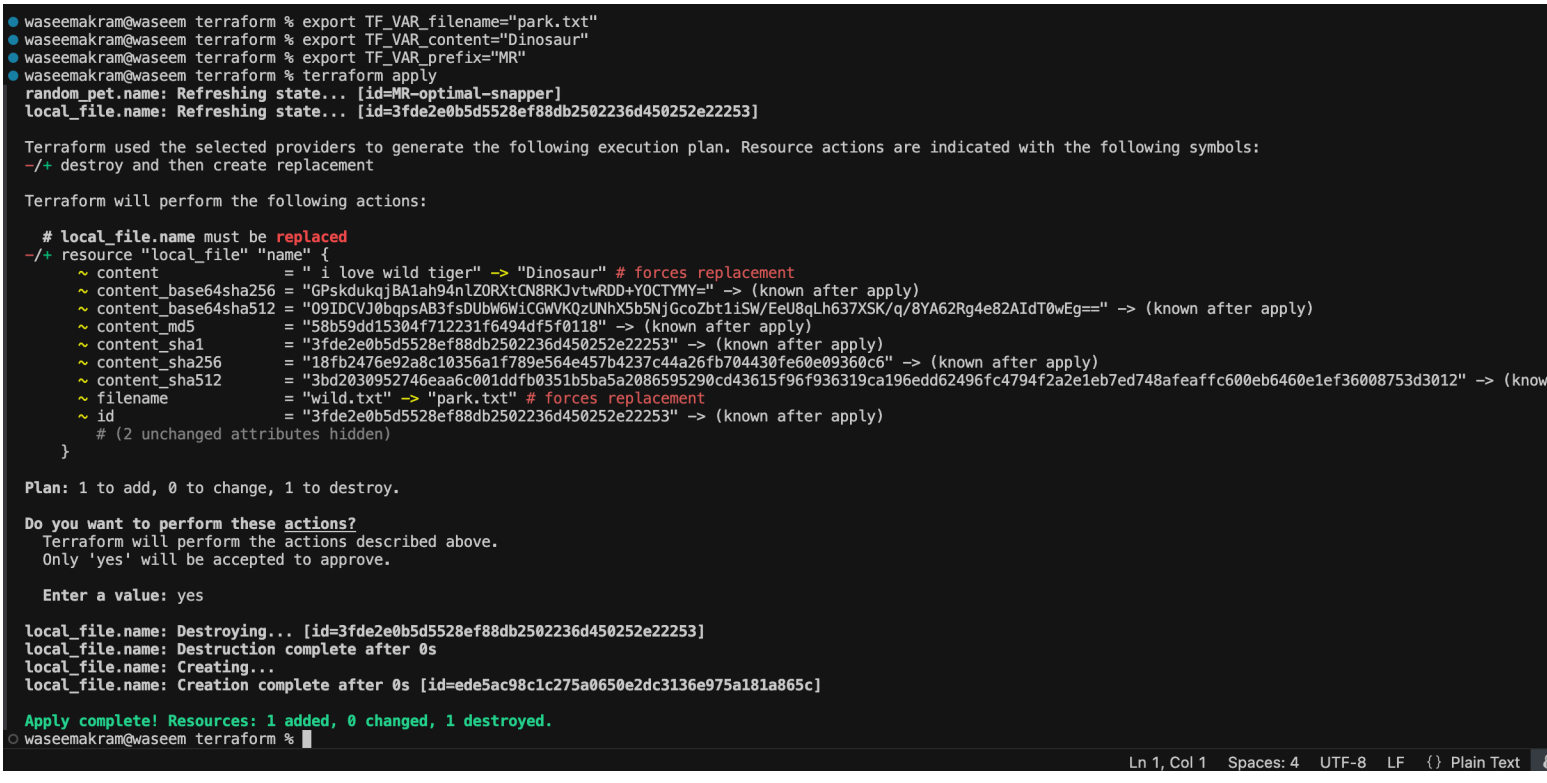
Mac (zsh) equivalent commands

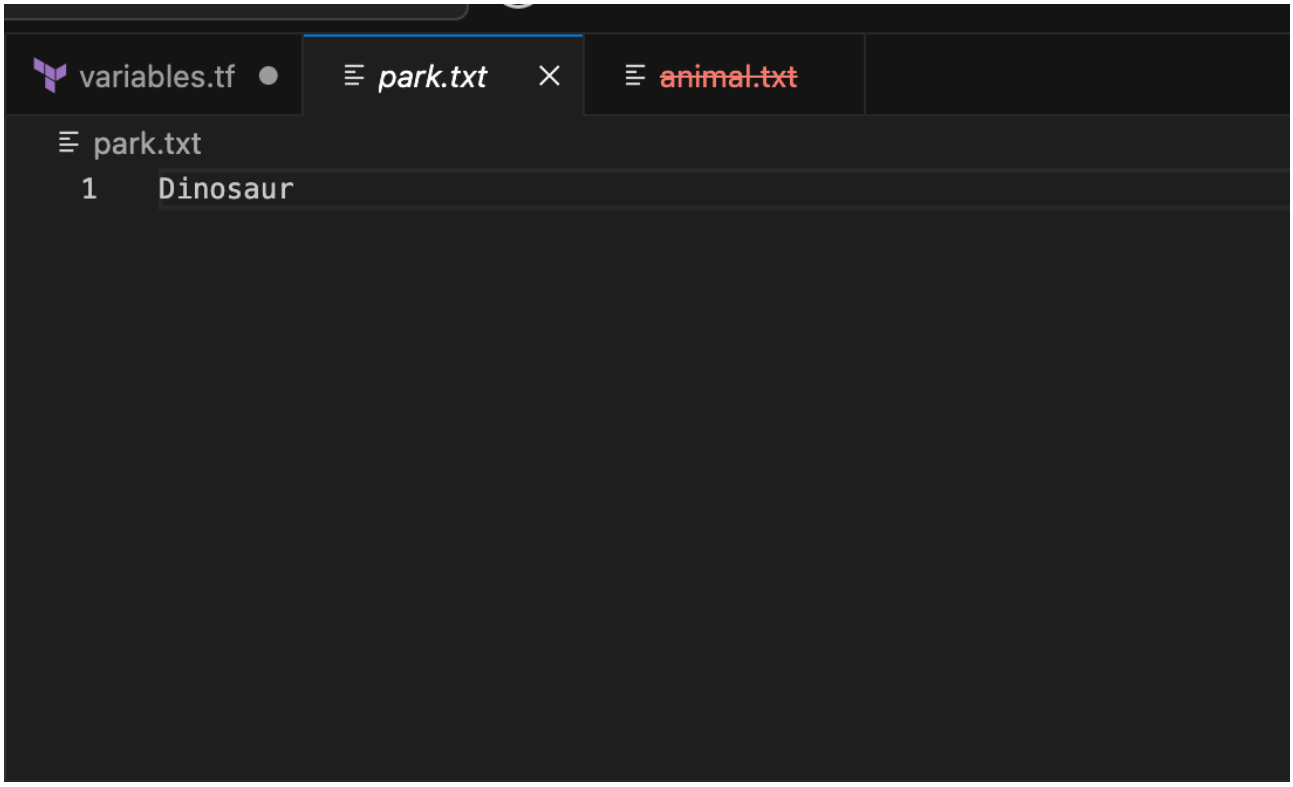
Set variables like this:

```
export TF_VAR_filename="park.txt"
export TF_VAR_content="Dinosaur"
export TF_VAR_prefix="MR"
```

Then run:

```
terraform apply
```





5) varibale definition file (Should be end with terraform.tfvars/ terraform.tfvars.json)

--

> for automatically loaded file name *.auto.tfvars/*.auto.tfvars.json

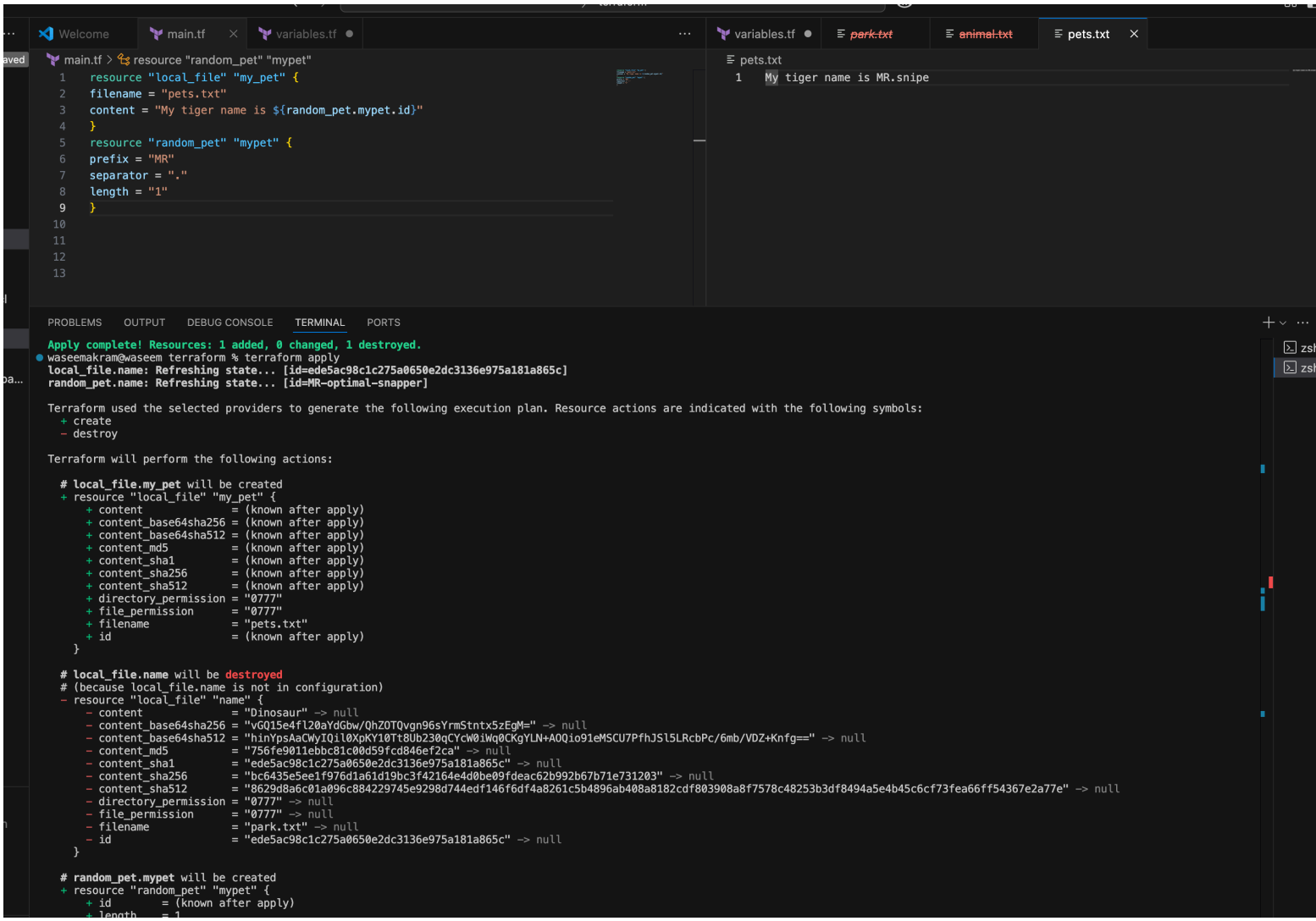
--

> if we are saving the file with other name like varible.tfvars then we need to pass this in

CLI

--

> terraform apply -var-file varibale.tfvars



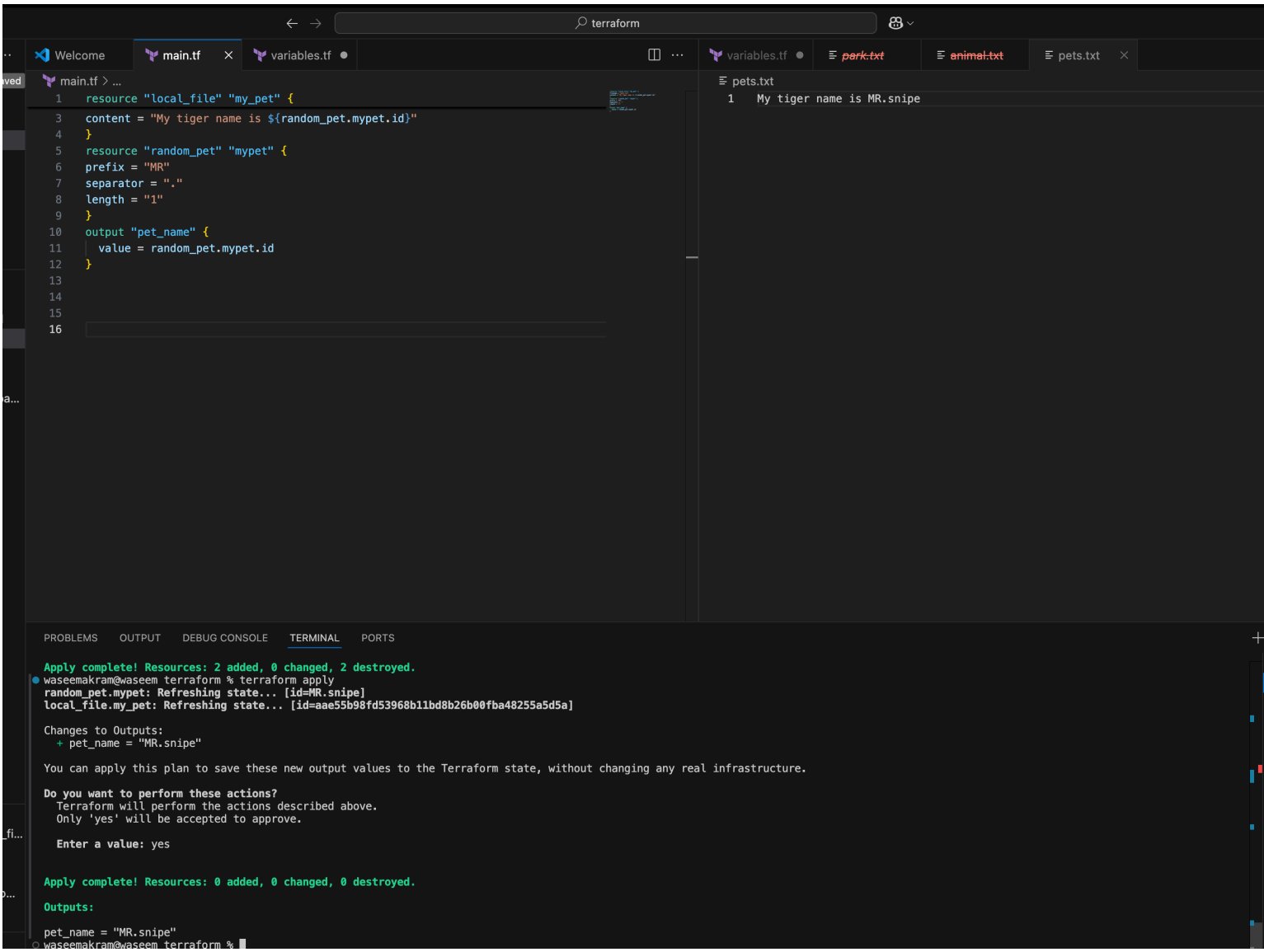
Output variable:

These are used to display the output of the resources.
Ex:
resource "random_pet" "mypet" {
prefix = "MR"
separator = "."
length = "1"
}

output my-pet {
value = random_pet.my-pet.id

description = optional name
}

when we use terraform apply we can see the id as output.
we can use terraform output command to see the output of the resource.



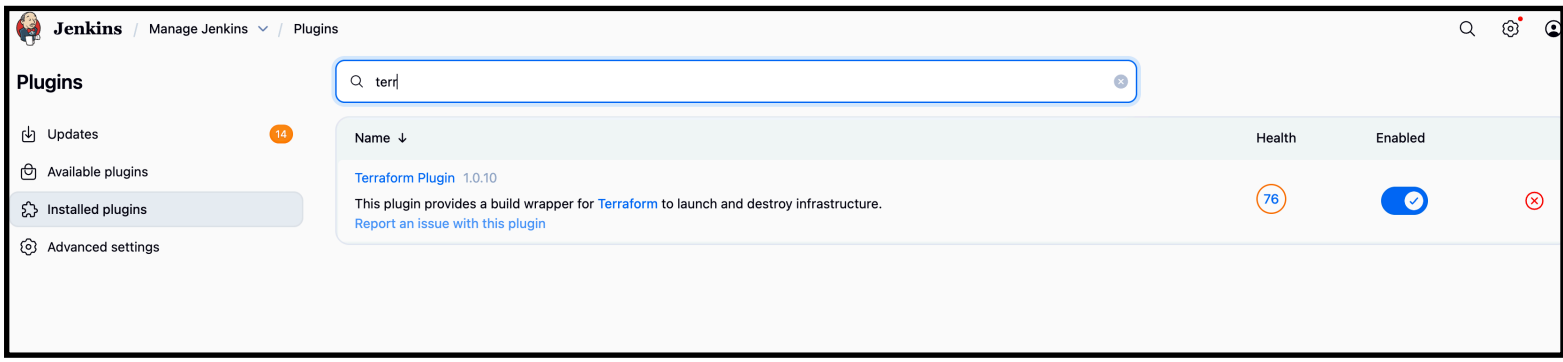
```
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

pet_name = "MR.snipe"
waseemakram@waseem terraform %
```

Task

3) Integrate Terraform in Jenkins using the Terraform plugin.

Step 1: Install the Terraform Plugin in Jenkins

1. Go to **Jenkins Dashboard → Manage Jenkins → Plugins → Available Plugins**.
2. Search for **Terraform**.
3. Install **Terraform Plugin**
4. Restart Jenkins if required.



Step 2: Configure Terraform in Jenkins

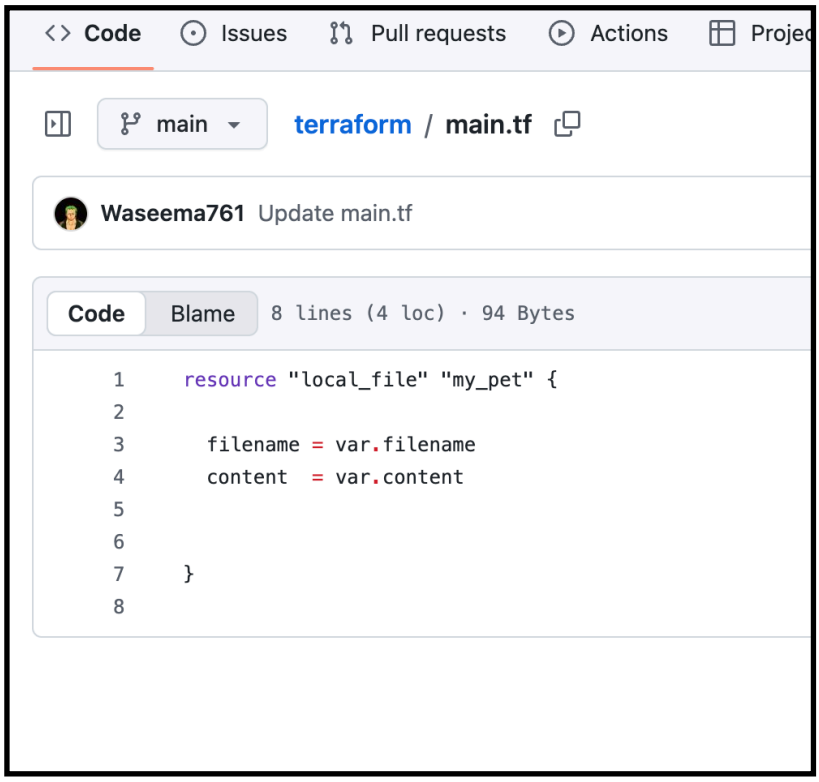
1. Go to **Manage Jenkins → Tools**.
2. Find **Terraform installations**.
3. Click **Add Terraform**:
    - Name: `terraform`
    - Install automatically: ✅ (Jenkins will download it, or you can specify the path if already installed).
4. Save.

Step 3: Create a Jenkins Job (Pipeline or Freestyle)

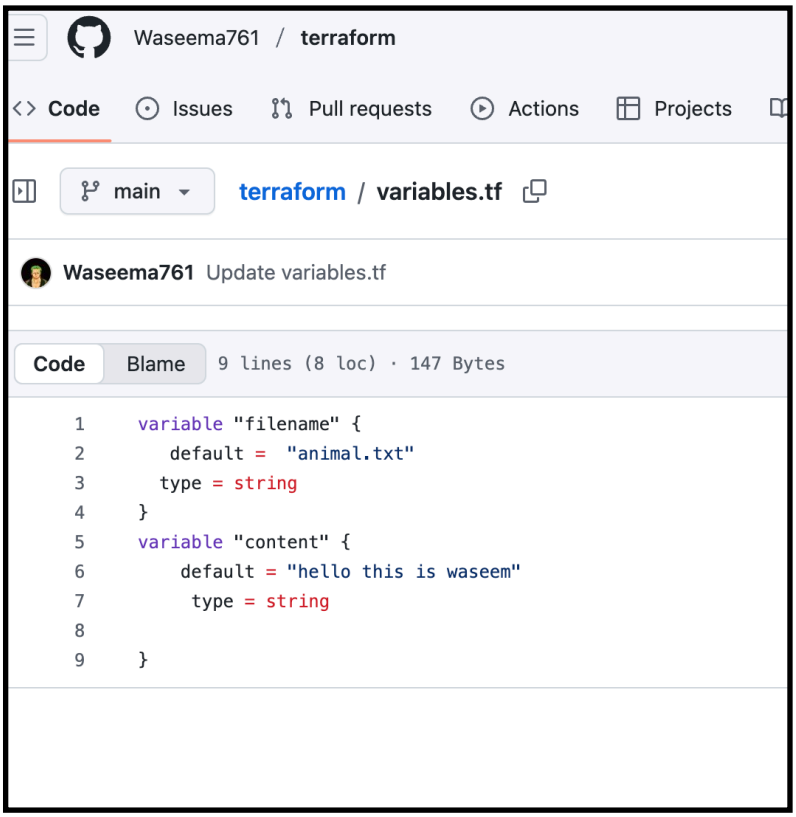
You can use either **Freestyle** or **Pipeline**.

—> inside GitHub

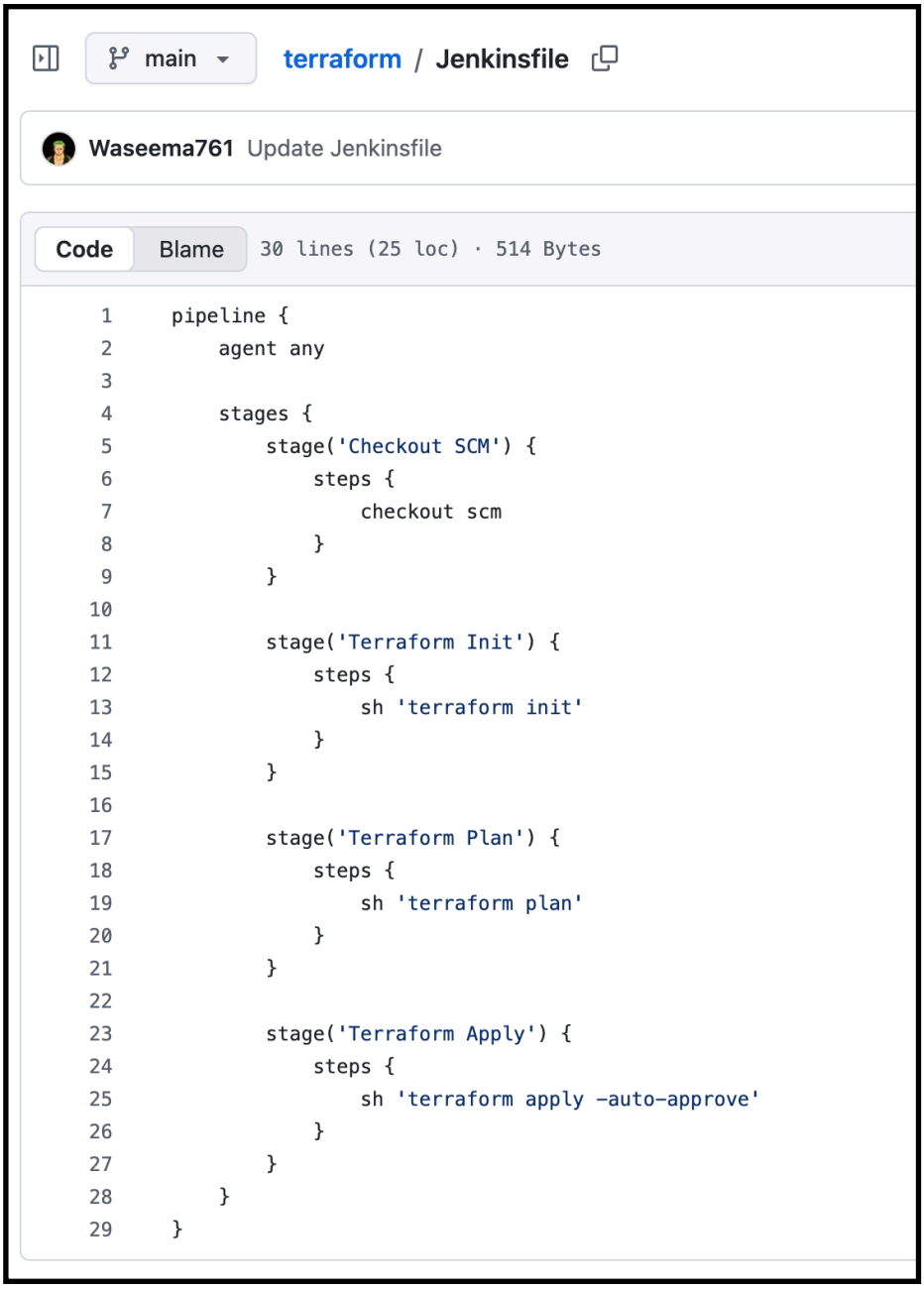
—> add this code in main.tf in your repo file



—> add this code in your variables.tf file



—> jenkinsfile



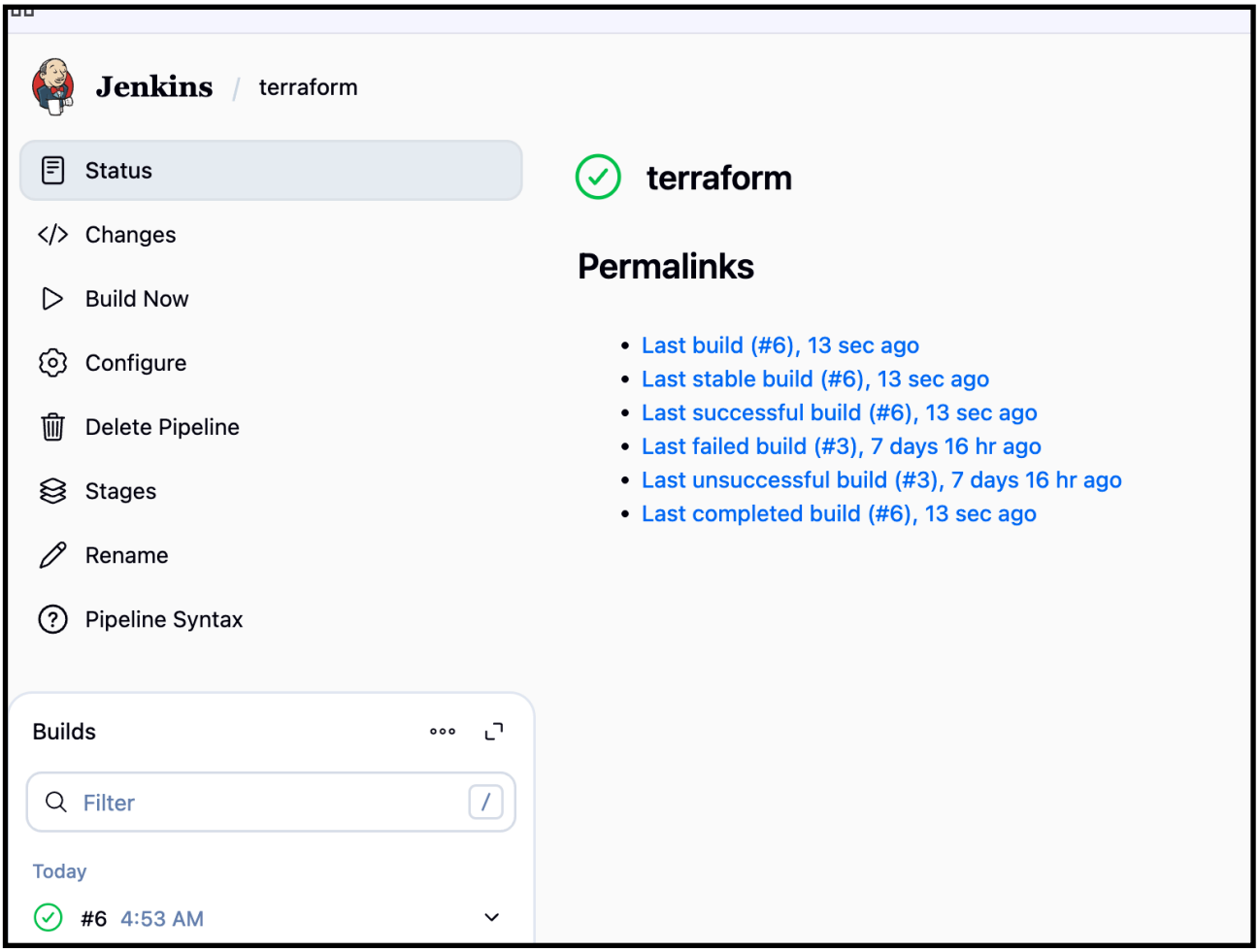
main terraform / Jenkinsfile

Waseema761 Update Jenkinsfile

Code Blame 30 lines (25 loc) · 514 Bytes

```
pipeline {
    agent any

    stages {
        stage('Checkout SCM') {
            steps {
                checkout scm
            }
        }

        stage('Terraform Init') {
            steps {
                sh 'terraform init'
            }
        }

        stage('Terraform Plan') {
            steps {
                sh 'terraform plan'
            }
        }

        stage('Terraform Apply') {
            steps {
                sh 'terraform apply -auto-approve'
            }
        }
    }
}
```

Now create a job and build



Check the Jenkins main workspace and cat the file inside the job you can view the content in the file

```
[ec2-user@ip-172-31-65-24 ~]$ cd /var/lib/jenkins/workspace/terraform
[ec2-user@ip-172-31-65-24 terraform]$ ls
Jenkinsfile  README.md  animal.txt  main.tf  pets.txt  terraform.tfstate  terraform.tfstate.backup  variables.tf
[ec2-user@ip-172-31-65-24 terraform]$
```

Task 4: Create one Jenkins job using Maven Project for the code below with two stages:

Stage 1: Git clone

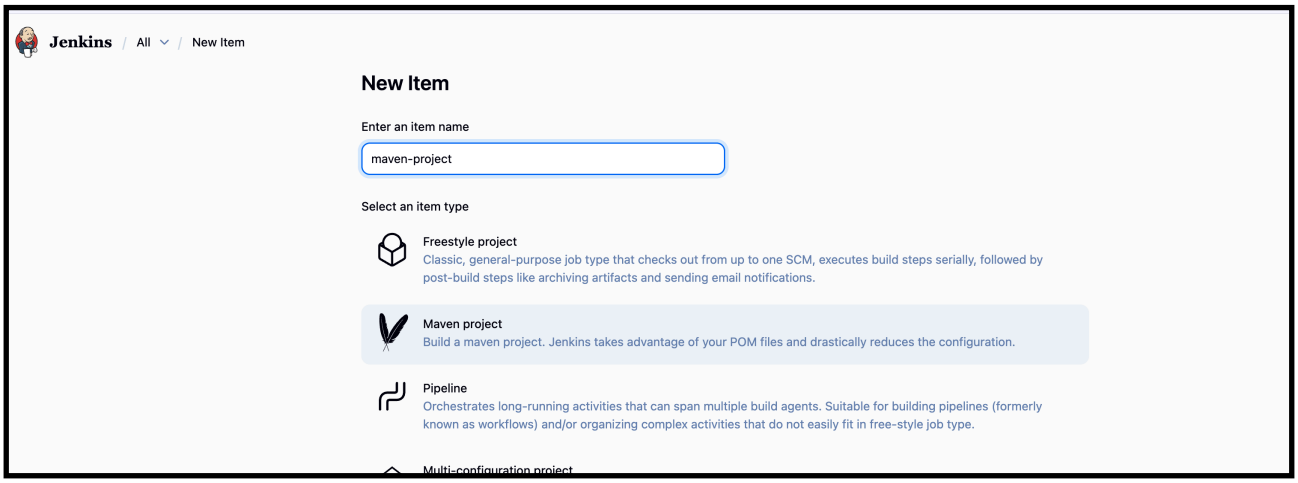
Stage 2: Maven Compilation Code: https://github.com/betawins/java-Working-app.git

Step-1

Install the **Maven Integration plugin** from **Manage Jenkins → Plugins**.

Step 2: create a job

Name it : maven-project
Select type : maven project

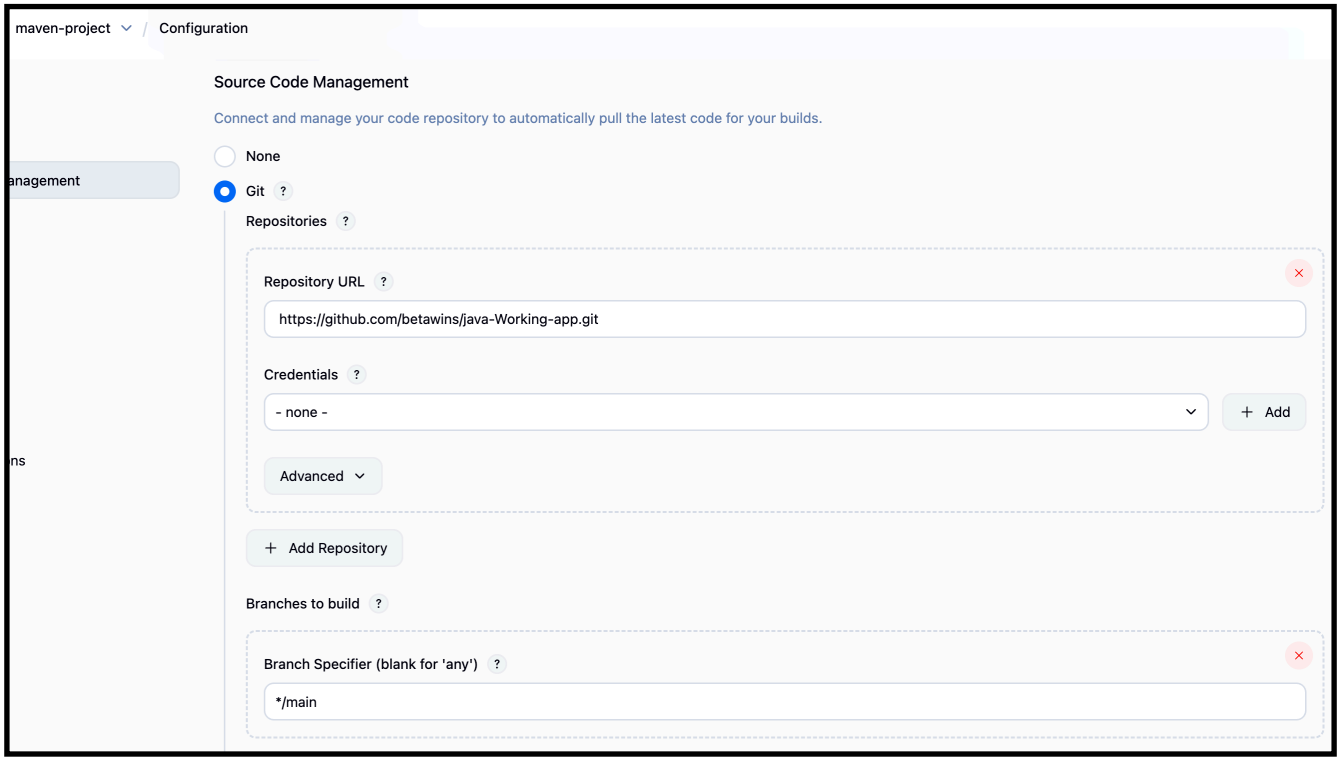


Step-3

Go to **Manage Jenkins → Tool Configuration → Maven**.

Add a Maven installation → check **"Install automatically"** → Jenkins downloads Maven when needed.

In **Source Code Management**, select **Git** and add your repo URL (**https://github.com/betawins/java-Working-app.git**).

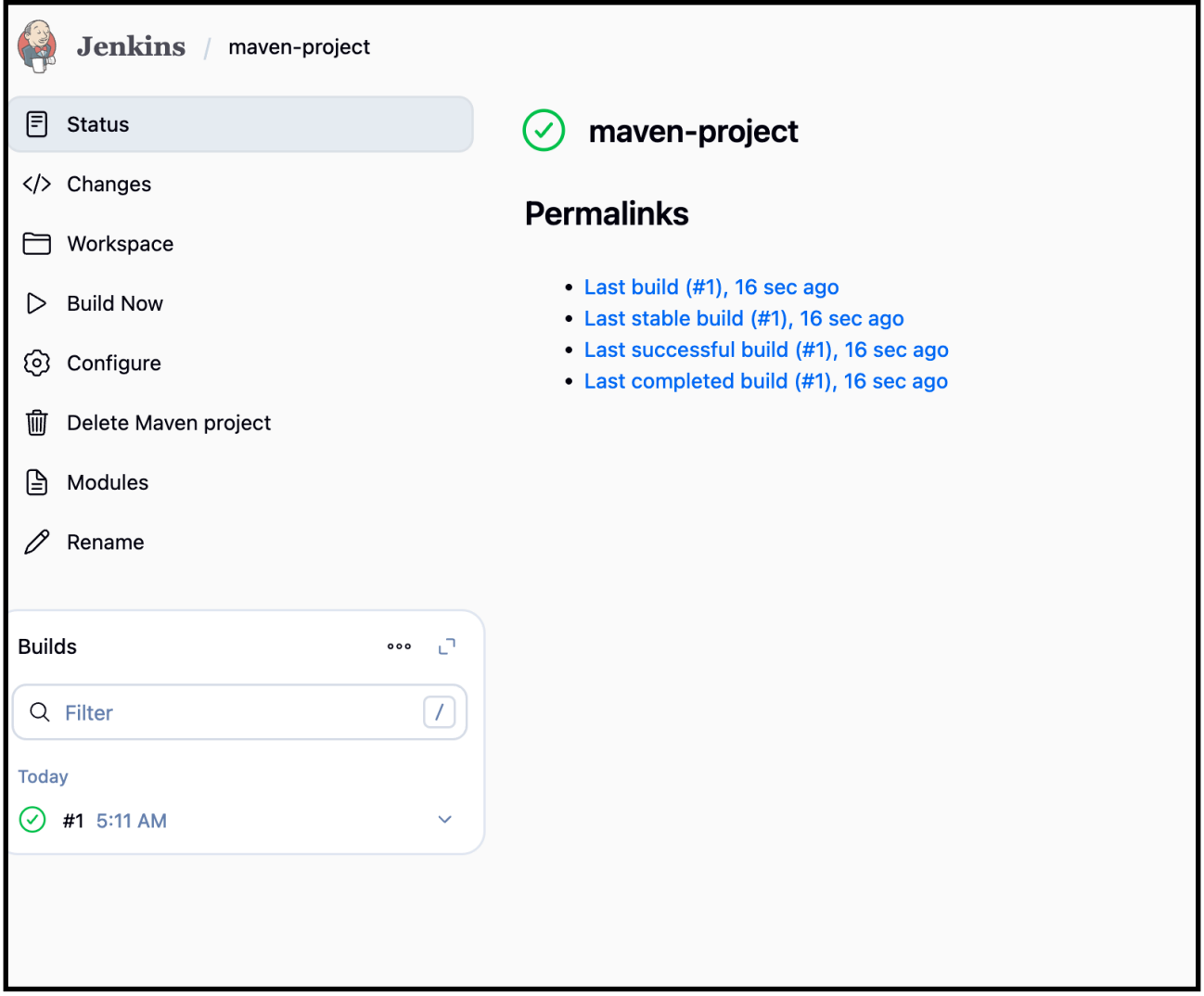


In **Build**, set:

- **Goals and options:** clean package
- Jenkins will run Maven using the installed Maven tool.

—> save

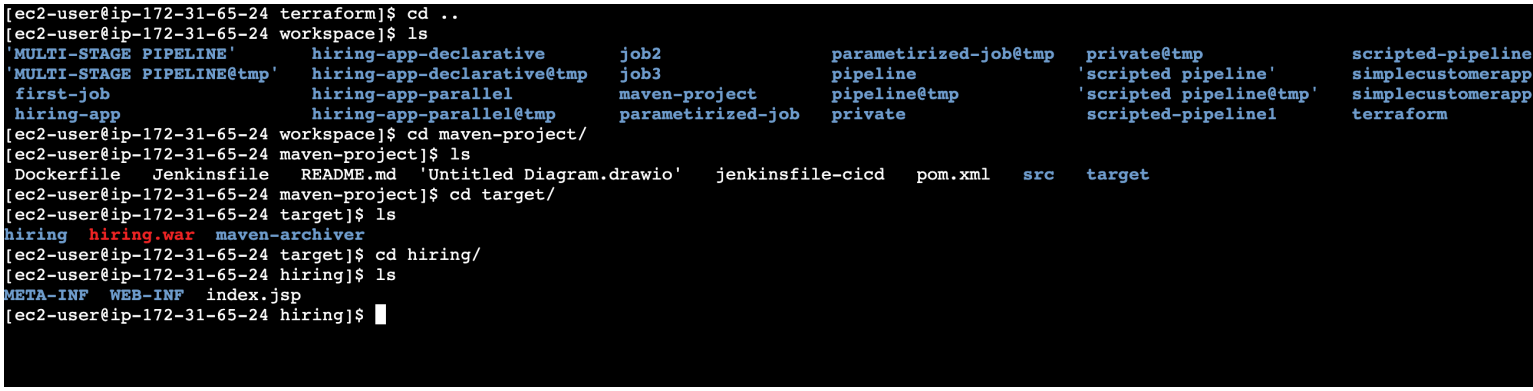
—-> now build and check output



Maven by default puts the output in the project's **target/ directory**.

Jenkins checks out your Git repo into a **workspace**:

/var/lib/jenkins/workspace/Maven/target/



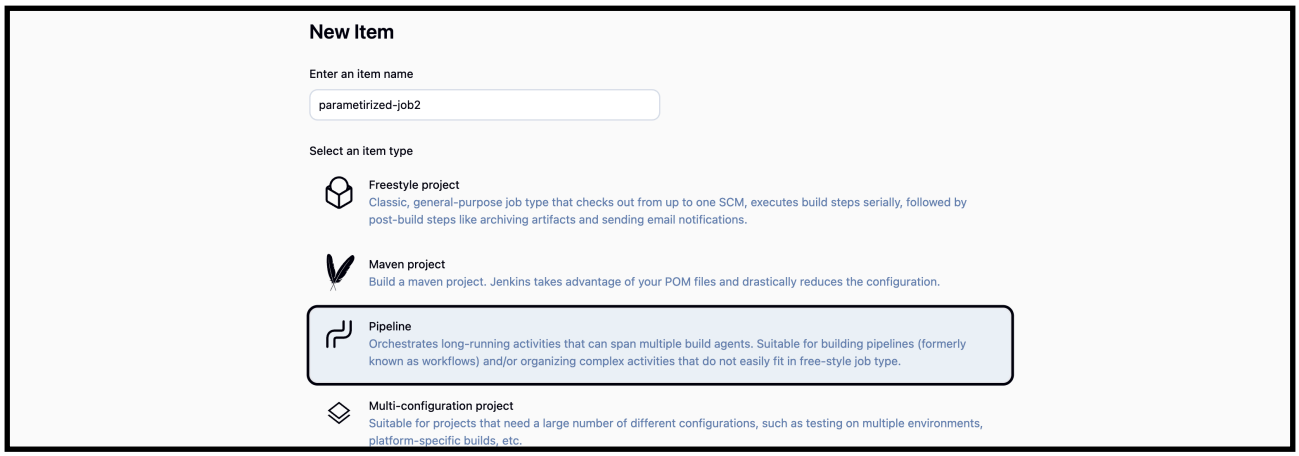
Task 5:

1. Use the same code and create a **parameterized job** in Jenkins with:
2. **Stage 1:** Git clone

Stage 2: Maven Compilation **Code:** https://github.com/betawins/java-Working-app.git

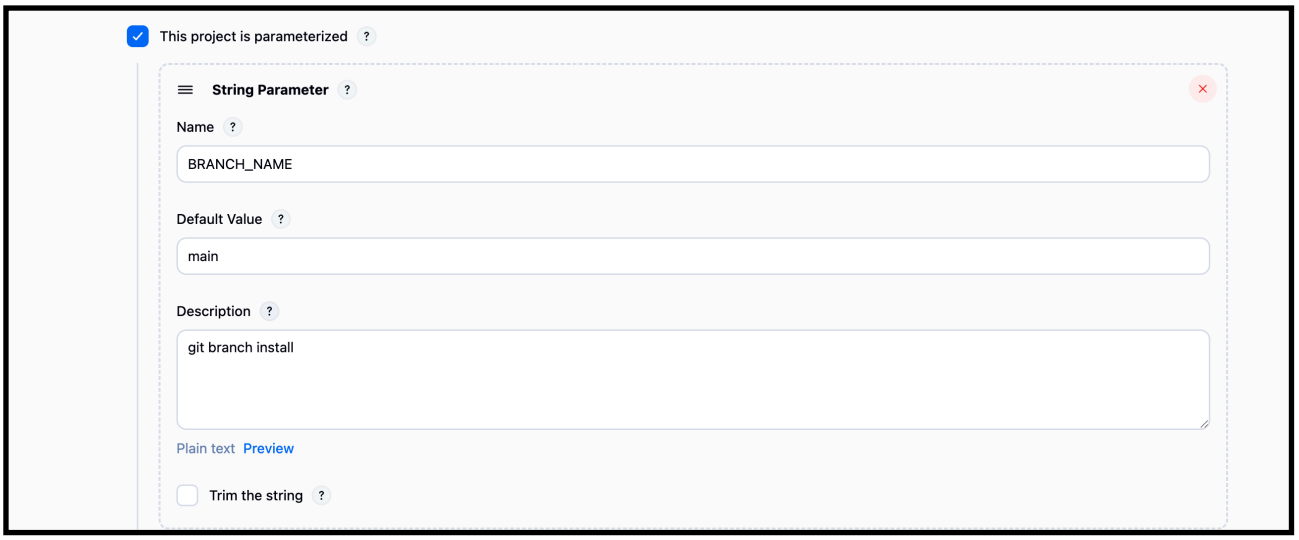
Step 1: Create a Parameterized Job

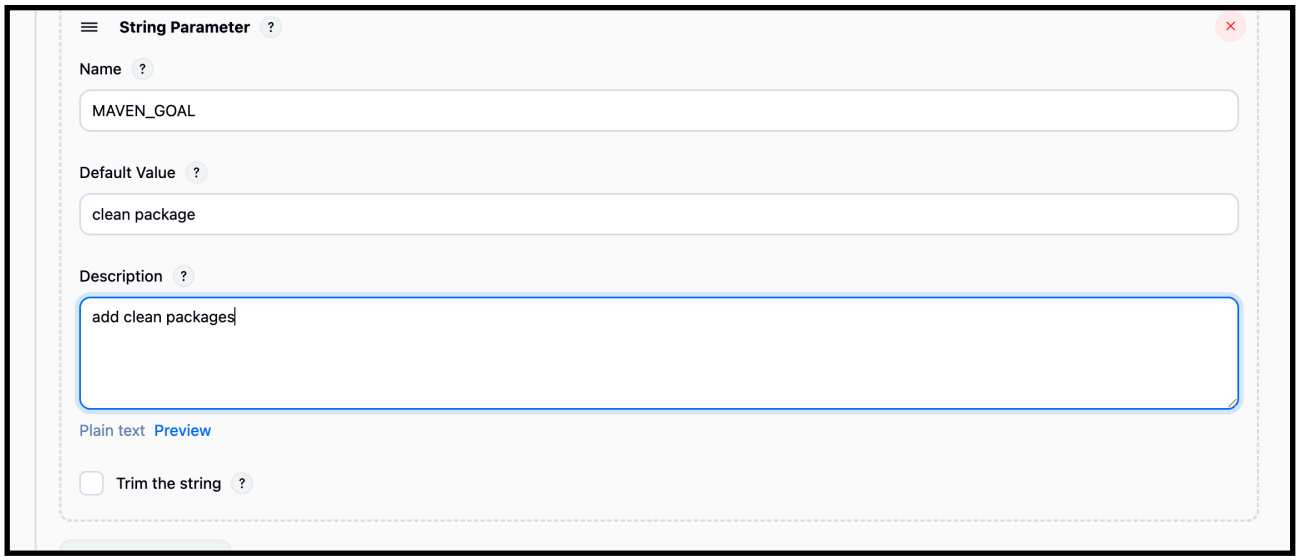
1. Go to **Jenkins Dashboard → New Item**
2. Choose **Pipeline** → give a name → click OK



In the job config:

- Check **“This project is parameterized”**
- Add parameters:
  - **String Parameter** → Name: BRANCH_NAME, Default: main, Description: Git branch to build
  - **String Parameter** → Name: MAVEN_GOAL, Default: clean package, Description: Maven goal to run





Step 2: Add Scripted Pipeline Code

```
node {
    def mvnHome = tool name: 'MVN_HOME', type: 'maven'
    env.PATH = "${mvnHome}/bin:${env.PATH}"

    stage('Git Clone') {
        git branch: "${BRANCH_NAME}", url: 'https://github.com/
betawins/java-Working-app.git'
    }

    stage('Maven Compilation') {
        sh "mvn ${MAVEN_GOAL}"
    }

    stage('Archive Package') {
        // Fix: WAR instead of JAR
        archiveArtifacts artifacts: 'target/*.war', fingerprint: true
    }
}
```

Definition

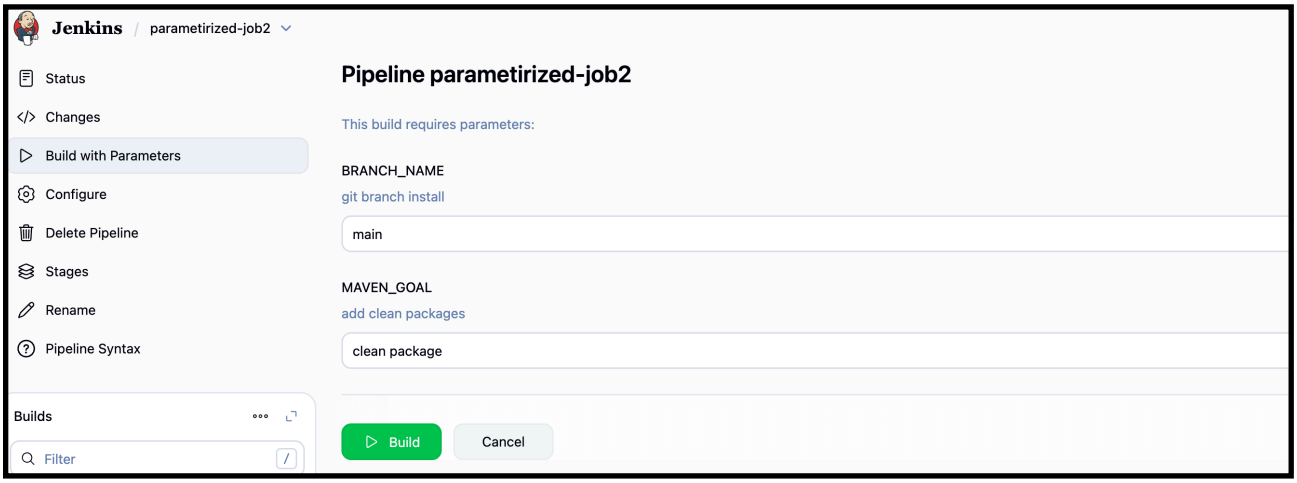
Pipeline script

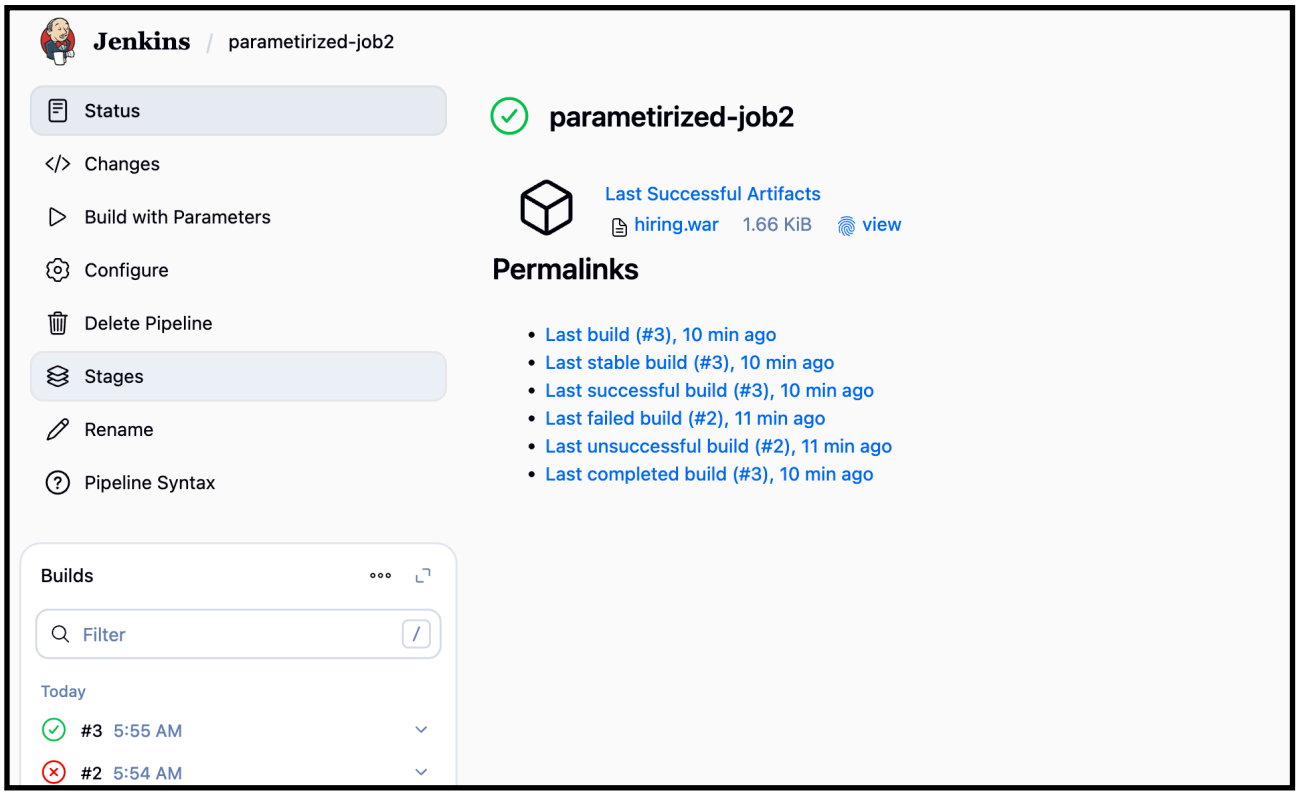
Script ?

```
node {
    def mvnHome = tool name: 'MVN_HOME', type: 'maven'
    env.PATH = "${mvnHome}/bin:${env.PATH}"

    stage('Git Clone') {
        git branch: "${BRANCH_NAME}", url: 'https://github.com/betawins/java-Working-app.git'
    }

    stage('Maven Compilation') {
        sh "mvn ${MAVEN_GOAL}"
    }

    stage('Archive Package') {
        // Fix: WAR instead of JAR
        archiveArtifacts artifacts: 'target/*.war', fingerprint: true
```

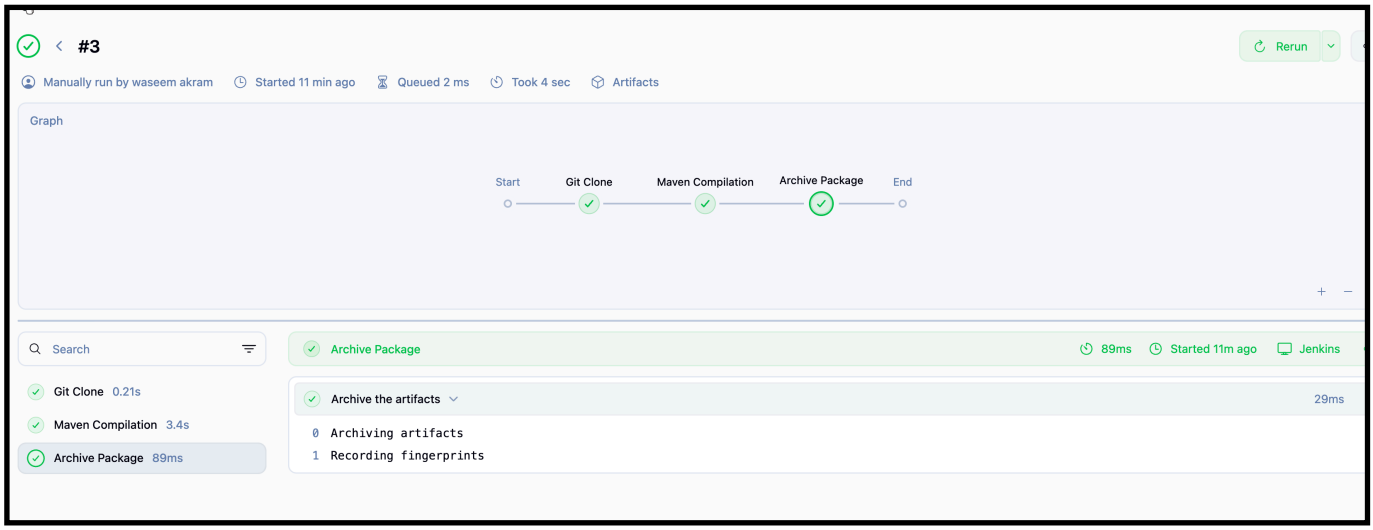
Use Groovy Sandbox ?

Save and build with parameters





—> check the pipeline stages



```
[ec2-user@ip-172-31-65-24 ~]$ cd /var/lib/jenkins/workspace/parametirized-job2
[ec2-user@ip-172-31-65-24 parametirized-job2]$ ls
 Dockerfile   Jenkinsfile   README.md  'Untitled Diagram.drawio'   jenkinsfile-cicd   pom.xml   src   target
[ec2-user@ip-172-31-65-24 parametirized-job2]$ cd target/
[ec2-user@ip-172-31-65-24 target]$ ls
hiring  hiring.war  maven-archiver
[ec2-user@ip-172-31-65-24 target]$
```

Task :
6.)What are the global variables in Jenkins?

In Jenkins Pipelines, **Global Variables** are special built-in environment variables and objects that Jenkins automatically provides. You don't need to define them — they are available everywhere in your pipeline.

Job & Build Info

- BUILD_ID → Unique build ID (timestamp)
- BUILD_NUMBER → Build number (incremental)
- BUILD_TAG → jenkins-${JOB_NAME}-${BUILD_NUMBER}
- JOB_NAME → Name of the Jenkins job
- JOB_BASE_NAME → Job name without folder path
- BUILD_URL → URL of the build in Jenkins
- JOB_URL → URL of the job

Node/Workspace Variables

- NODE_NAME → Name of the agent node running the build
- WORKSPACE → Path to the job's workspace directory

Git/SCM Variables (when using git step)

- GIT_COMMIT → Commit hash being built
- GIT_BRANCH → Branch being built
- GIT_URL → Git repo URL

Special Global Objects (Pipeline DSL)

These are not strings, but powerful objects you can call methods on:

- steps → Provides pipeline steps (e.g. sh, git, echo)
- pipeline → DSL for declarative pipelines
- docker → Access Docker agents/containers if plugin is enabled
- input → Request user input

- credentials() → Inject stored credentials

Summary

- **env** → Environment variables
- **params** → Build parameters
- **currentBuild** → Build metadata
- **Built-in vars** → BUILD_NUMBER, JOB_NAME, WORKSPACE, BUILD_URL, etc.
- **SCM vars** → GIT_COMMIT, GIT_BRANCH, GIT_URL